

Documentación Completa del Proyecto: AutoCare

1. Información General del Proyecto

- **Nombre del Proyecto:** AutoCare
- **Descripción:** Sistema de microservicios para la gestión integral de un taller mecánico automotriz. La arquitectura está diseñada con alta cohesión, permitiendo la escalabilidad independiente de cada módulo y garantizando la resiliencia operativa.
- **Autores/Integrantes:** Fernando Barra, Benjamín Montanares, Sebastián Saavedra.

2. Tecnologías y Stack Técnico

El proyecto se basa en las siguientes tecnologías modernas:

- **Lenguaje:** Java 21 (Uso extensivo de Records para la inmutabilidad de datos).
- **Framework Principal:** Spring Boot 3.5.14.
- **Ecosistema Cloud:** Spring Cloud (Eureka Discovery Client para registro, API Gateway para enrutamiento, LoadBalancer).
- **Persistencia de Datos:** JPA, Hibernate y PostgreSQL. Cada microservicio tiene una base de datos independiente y aislada (`ddl-auto: update`).
- **Comunicación Interservicios:** Spring WebFlux (`WebClient`) para comunicación síncrona entre servicios.
- **Validaciones:** Bean Validation (Jakarta).
- **Herramientas adicionales:** Lombok (para reducir el código boilerplate).

3. Arquitectura y Microservicios

El ecosistema consta de varios microservicios con responsabilidades únicas (Single Responsibility Principle). El hilo conductor entre casi todos los servicios es el identificador del vehículo (`id_vehiculo` o patente).

3.1. Infraestructura Core

- `eureka-server` (**Puerto 8761**): Servidor de registro y descubrimiento de servicios. Mantiene el "mapa estelar" de dónde se encuentra cada servicio.
- `api-gateway` (**Puerto 8080**): Puerta de entrada única y enrutamiento hacia todos los demás servicios.

3.2. Microservicios de Negocio

1. **garage-service** (Puerto 8081 - Gestión de Vehículos y Clientes):

- **Responsabilidad:** Gestión unificada de clientes y sus vehículos (Fleet & Customer Service). Dueño de la información técnica del auto (patente, VIN, marca) y datos de contacto del cliente.
- **Ruta Gateway:** `/api/garage/**`

2. **inventory-service** (Puerto 8082 - Inventario y Repuestos):

- **Responsabilidad:** Control de stock de bodega y catálogo de repuestos (Spare Parts Service). Permite reservar piezas para un presupuesto.
- **Ruta Gateway:** `/api/inventario/**`

3. **hr-service** (Puerto 8083 - Gestión de Staff):

- **Responsabilidad:** Gestión del personal (técnicos/mecánicos), especialidades y su disponibilidad en tiempo real.
- **Ruta Gateway:** `/api/personal/**`

4. **billing-service** (Puerto 8084 - Facturación):

- **Responsabilidad:** Emisión de facturas, montos, impuestos y control de pagos. Cierre comercial de la atención.
- **Ruta Gateway:** `/api/facturacion/**`

5. **booking-service** (Puerto 8085 - Agenda y Citas):

- **Responsabilidad:** Agendamiento de citas, control del calendario de ingresos y validación de disponibilidad de horarios.
- **Ruta Gateway:** `/api/reservas/**`

6. **workshop-service** (Puerto 8086 - Órdenes de Trabajo / Workflow):

- **Responsabilidad:** Núcleo de operaciones. Mueve el auto por el taller mediante órdenes de trabajo (En Espera, En Proceso, Listo).
- **Ruta Gateway:** `/api/taller/**`

7. **diagnostics-service** (Puerto 8087 - Diagnóstico y Presupuesto):

- **Responsabilidad:** Historial clínico del vehículo, telemetría, códigos OBD2, cálculo de presupuestos y horas-hombre estimadas.
- **Ruta Gateway:** `/api/diagnosticos/**`

8. **loyalty-service** (Puerto 8088 - Lealtad):

- **Responsabilidad:** Sistema de puntos, niveles y recompensas para clientes recurrentes.
- **Ruta Gateway:** `/api/lealtad/**`

9. **procurement-service** (Puerto 8089 - Compras):

- **Responsabilidad:** Gestión de proveedores y abastecimiento (órdenes de compra de repuestos).
- **Ruta Gateway:** `/api/compras/**`

10. **analytics-service** (Puerto 8090 - Métricas):

- **Responsabilidad:** Observatorio de métricas, reportes financieros y estadísticos del taller.

- **Ruta Gateway:** `/api/metricas/**`

11. `notification-service` (Puerto 8091 - Notificaciones / CRM):

- **Responsabilidad:** Envío y gestión de avisos al cliente por SMS/WhatsApp o correo (ej. "Tu auto está reparado").
- **Ruta Gateway:** `/api/notificaciones/**`

4. Comunicación Interservicios (WebClient)

La independencia de las bases de datos obliga a los servicios a comunicarse por red usando llamadas HTTP síncronas (`WebClient`). Ejemplos destacados:

- `garage-service` → `loyalty-service` : Al registrar un cliente nuevo en garage, llama a loyalty para crear automáticamente una cuenta de puntos con saldo cero.
- `workshop-service` → `inventory-service` : Cuando un mecánico usa una pieza física, se descuenta el stock en tiempo real llamando al inventario.
- `analytics-service` → `billing-service` : Para generar el reporte mensual, consulta los ingresos efectivos.
- `booking-service` → `garage-service` : Antes de confirmar una cita, verifica la existencia real del cliente y su vehículo mediante sus IDs.

5. Reglas de Negocio Centrales (Enfocadas en `booking-service`)

El módulo de citas posee reglas de negocio (RN) estrictas procesadas en su capa `CitaService` :

- **RN-01 (Existencia):** Se verifica contra `garage-service` que el vehículo y cliente existan; si no, retorna error 400.
- **RN-02 (Duración):** Toda cita dura **60 minutos** estándar (usado para calcular bloques ocupados).
- **RN-03 (Sin Choques de Horario):** Un vehículo no puede tener dos citas en estado CONFIRMADA o EJECUTADA que se solapen en tiempo.
- **RN-04 (Límite Diario):** El taller acepta un máximo de **20 citas por día**. Excederlo lanza excepción de negocio.
- **RN-05 (Transiciones de Estado):** El ciclo de vida de una cita es: `AGENDADA` → `CONFIRMADA` o `CANCELADA` . De `CONFIRMADA` pasa a `EJECUTADA` o `CANCELADA` . (Cancelada y Ejecutada son estados finales).
- **RN-06 (Auto-creación de Orden):** Cuando la cita pasa a `EJECUTADA` (el auto llega físicamente), se llama a `workshop-service` para abrir una "Orden de Trabajo" en estado `RECEPCIONADO` .
- **RN-07 (Inmutabilidad Temporal):** No se pueden crear citas en el pasado (validación `@Future`).
- **RN-08 (Días Hábiles):** No se permiten agendamientos durante los fines de semana (sábado y domingo).

6. Pruebas Unitarias y Calidad

El proyecto tiene un fuerte enfoque en calidad de software, especificado en su plan de pruebas:

- **Herramientas:** JUnit 5, Mockito (para mockear repositorios JPA y WebClient), JaCoCo (para medir cobertura).
- **Estructura de Tests:** Utiliza el patrón **Given-When-Then** (AAA: Arrange-Act-Assert) para máxima claridad. Aísla las reglas de negocio en la capa Service sin tocar bases de datos reales.
- **Cobertura Mínima:** El `booking-service` exige una cobertura de líneas $\geq 80\%$ y ramas $\geq 70\%$, alcanzando un $\geq 90\%$ en la clase crítica `CitaService`.

7. Despliegue e Infraestructura (Evaluación Parcial 3)

- El módulo crítico `booking-service` ha sido preparado para despliegue remoto en plataformas como **Railway o Render**.
- La base de datos es un servidor **PostgreSQL Cloud** dedicado.
- Se utilizan variables de entorno inyectadas en la plataforma cloud para proteger secretos (credenciales de DB), manteniendo la seguridad y separación de configuraciones (Factor de 12-factor apps).

8. Justificación de la Arquitectura (Frente a un Monolito)

Para defender el diseño ante evaluaciones técnicas:

- **Escalabilidad Independiente:** Si el taller recibe masivas consultas de citas web, se pueden levantar más instancias de `booking-service` sin desperdiciar RAM en el servicio de facturación.
- **Resiliencia y Aislamiento de Fallos (Independencia de datos):** Si la base de datos del inventario de repuestos se corrompe o cae, los mecánicos aún pueden actualizar las Órdenes de Trabajo (`workshop-service`) ya que cada servicio tiene su propia BD y no hay caídas en cascada totales.
- **Libertad Tecnológica:** Cada servicio puede ser escrito en el lenguaje más óptimo para su tarea (ej. Diagnóstico predictivo en Python, Facturación en Java) sin fricción en el código.